

CSS Position Property — Practice Problems

Topics Covered

1. `position: static`
 2. `position: relative` + `position: absolute`
 3. `position: fixed` — Floating Help Button
 4. `position: fixed` — Fixed Navbar
 5. `position: sticky` — Sticky Section Headings
 6. `position: sticky` — Sticky Table Header
-

Problem 1 — `position: static`

Student Profile Page

Create a **Student Profile Page** containing the following elements:

- Profile Image
- Student Name
- Course
- Email
- Short Description
- Contact Button

Requirements

1. Use:

```
position: static;
```

2. All elements should appear in their **normal document flow**.

3. Do **not** use:

- `top`
- `right`
- `bottom`
- `left`

to arrange the elements.

4. Use `margin` and `padding` for proper spacing.

5. Try applying:

```
top: 50px;
```

to the student name and observe what happens.

Goal

Understand that:

- `static` is the default position.
- Elements remain in the normal document flow.
- `top`, `right`, `bottom`, and `left` do not affect a static element.

Problem 2 — `relative` + `absolute`

Product Card

Create a **Product Card** similar to the following:



Requirements

Card

The product card must use:

```
position: relative;
```

SALE Badge

The `SALE` badge must use:

```
position: absolute;
```

Place it at the **top-left** corner of the card.

Heart Icon

The heart icon must use:

```
position: absolute;
```

Place it at the **bottom-right** corner of the card.

Additional Requirements

- Create a proper product card design.
- Give the card a fixed width and height.
- Add a border and border-radius.
- Add suitable padding.
- Center the product image.
- Display the product name and price normally.

Goal

Understand how:

```
relative parent  
+  
absolute children
```

are used to position elements inside a component.

Problem 3 — position: fixed

Floating Help Button

Create a webpage for an **Online Course Platform**.

The webpage should contain:

- Navbar
- Course Description
- Course Modules
- Instructor Information

- Student Reviews
- Footer

The page should contain enough content so that the user can scroll.

Floating Help Button

Add a floating help button:



Requirements

The button must:

1. Stay at the **bottom-right** of the screen.
2. Remain visible while scrolling.
3. Be **20px** from the right.
4. Be **20px** from the bottom.
5. Use:

```
position: fixed;
```

Expected Position

```
.help-button {  
  position: fixed;  
  right: 20px;  
  bottom: 20px;  
}
```

Challenge

Make the help button:

- Circular
- Large enough to click easily
- Visually different from the normal page content

Goal

Understand that **fixed** positions an element relative to the **browser viewport** and keeps it visible while scrolling.

Problem 4 — `position: fixed`

Fixed Navbar

Create a webpage with the following navbar:

```
+-----+  
| LOGO | HOME | COURSES | ABOUT | LOGIN |  
+-----+
```

Below the navbar, create a long webpage containing:

```
CONTENT  
  
CONTENT  
  
CONTENT  
  
CONTENT  
  
CONTENT  
  
CONTENT
```

Add enough content so that the page can be scrolled.

Requirements

1. The navbar must remain at the **top of the screen** while scrolling.
2. The navbar must span the **entire width** of the page.
3. Use:

```
position: fixed;
```

4. The navbar should remain visible even when the user scrolls down.
5. Make sure the page content does **not hide behind the navbar**.

Important

Because the navbar is fixed, it is removed from the normal document flow.

Therefore, add suitable spacing to the content:

```
main {  
  padding-top: /* navbar height */;  
}
```

Challenge

Add a:

Back to Top ↑

button at the bottom-right of the screen.

The button should also use:

```
position: fixed;
```

Final Layout

```
+-----+  
| LOGO | HOME | COURSES | ABOUT | LOGIN |  
+-----+  
  
PAGE CONTENT  
  
PAGE CONTENT  
  
PAGE CONTENT  
  
PAGE CONTENT  
  
+-----+  
|   ↑   |  
| Back Top |  
+-----+
```

Goal

Understand how `position: fixed` can be used to create:

- Fixed navigation bars
- Floating buttons
- Back-to-top buttons
- Persistent UI elements

Problem 5 — position: sticky

Sticky Section Headings

Create a webpage containing three sections:

STUDENTS

Student 1
Student 2
Student 3
Student 4
Student 5
Student 6
Student 7
Student 8
Student 9
Student 10

TEACHERS

Teacher 1
Teacher 2
Teacher 3
Teacher 4
Teacher 5
Teacher 6
Teacher 7
Teacher 8
Teacher 9
Teacher 10

COURSES

Java
Python
C
C++
JavaScript
React
Node.js
SQL

Requirements

1. Make each section heading sticky.
2. The heading should remain at the top while scrolling through its section.
3. Use:

```
position: sticky;  
top: 0;
```

4. Give each section enough content so that scrolling is required.

Example

```
.section-title {  
  position: sticky;  
  top: 0;  
}
```

Challenge

Make the heading visually different when it becomes sticky.

For example:

- Add a background
- Add padding
- Add a shadow
- Change the font size
- Add a border

Goal

Understand that **sticky**:

```
Normal Position  
  ↓  
User Scrolls  
  ↓  
Reaches top: 0  
  ↓  
Element Sticks  
  ↓  
Section Ends  
  ↓  
Element Leaves
```

Problem 6 — position: sticky

Sticky Table Header

Create a **Student Marks Dashboard** containing a table.

Name	Course	Marks	Grade
Rahul	Java	85	A
Amit	Python	78	B
Neha	C++	92	A+
Raj	Java	67	C
Priya	Python	88	A
...			

Requirements

1. Create at least **20 students**.
2. The table must contain:
 - Student Name
 - Course
 - Marks
 - Grade
3. The table should be long enough to require scrolling.
4. Make the table header sticky.
5. Use:

```
position: sticky;  
top: 0;
```

6. The table header must remain visible while scrolling through the table.

Example

```
th {  
  position: sticky;  
  top: 0;  
}
```

Challenge — Dashboard Version

Create a dashboard containing:

+-----+				
	NAVBAR			
+-----+				
+-----+				
	Dashboard			
+-----+				
+-----+				
	Name		Course	
	Marks		Grade	
+-----+				
	Rahul		Java	
	85		A	
	Amit		Python	
	78		B	
	...			

The navbar should remain fixed at the top.

The table header should become sticky **below the navbar**.

For example:

```
.table-header {  
  position: sticky;  
  top: 60px;  
}
```

where 60px represents the navbar height.

Goal

Understand a real-world use of sticky positioning in:

- Student dashboards
- Admin panels
- Data tables
- Reports
- Financial dashboards
- Management systems

CSS Position — Quick Revision

Position	Main Purpose
static	Normal document flow
relative	Move element / create reference for absolute children

Position	Main Purpose
<code>absolute</code>	Position element inside a positioned parent
<code>fixed</code>	Attach element to viewport
<code>sticky</code>	Stick element during scrolling

Remember

`static`

Normal Flow

`relative`

Normal Flow
+
Can be moved
+
Creates positioning context

`absolute`

Removed from Normal Flow
+
Positioned relative to nearest positioned ancestor

`fixed`

Removed from Normal Flow
+
Attached to Viewport

`sticky`

Normal Flow
+

Sticks during Scrolling

Important Rule

The most important relationship to remember is:

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
}
```

This pattern is extremely common in real-world CSS for:

- Product cards
- Notification badges
- Image overlays
- Icons
- Close buttons
- Labels
- Dropdown indicators
- UI components